

Past, Current and Future in Data Engineering

Prepared for

Dr. Aryati Bakri

Senior Lecturer, Faculty of Computing, University Technology Malaysia

By

Muhammad Afiq Danial Bin Rozaidie¹ and Ahmad Adib Zikri bin A.Mazlam²

Students, Faculty of Computing, University Technology Malaysia

April 12, 2026

I. Executive Summary - afiq

This paper explores the evolution of data engineering with focus on transition from using traditional database systems or RDBMS and batch processing into modern, scalable, real-time data streaming while being able to load large amounts of data. The aim of this study is to analyse between past, current and future developments of data engineering while investigating its technologies, challenges and future trends that give a massive impact to data-driven decision making. The paper concludes that data engineering plays a crucial role towards the organization to adopt proper governance, structured modeling, and suitable technologies.

II. Introduction

Data engineering has become such a hot topic in the current computational era especially when almost every company and organization uses it in a data and technology driven field. The definition of data engineering is the development, implementation, and maintenance of systems and processes that take in raw data and produce high-quality, consistent information that supports downstream use cases, such as analysis and machine learning. Data engineering is the intersection of security, data management, DataOps, data architecture, orchestration, and software engineering (Reis, J., & Housley, M., 2022). Another study from Rucco et al. (2025) states that data engineering involves designing and building systems for collecting, storing, and analyzing data at scale, ensuring that data is accessible, reliable, and ready for analysis.

Data engineering has been established since the 1970s with the foundation practices of relational database and Extract, Transform, Load (ETL) processes. Although it has evolved significantly, many organizations still face some challenges while maintaining data efficiently during the different stages of development. Some issues like data consistency, poor data quality, lack of scalability and difficulties with integration from a different platform are the usual struggle that most organizations have to deal with.

The objective of this paper is to investigate and analyze the historical development and tools of data engineering from past, present and future while also providing insights on how data engineering can better support data-driven decision making as time continues to advance. The scope of this study is to focus on the evolution of data engineering and also understanding the whole concept, trend and challenges.

III. Background/Literature Review

Early data engineering systems mainly used a Relational Database Management Systems (RDBMS) as a database model that allowed structured data storage and efficient querying data using the SQL. Studies from Hegde (2024) states that data management traces back its monolithic roots in the early days with RDBMS as the leader of the landscape-Oracle, Microsoft SQL Server, and MySQL. These systems may sound reliable but not well suited for large analytical operations. During this period, most organizations strongly used ETL

pipelines to extract data from operational systems, transformed and then loaded into data warehouses to make a report and analyse out of it. But these processes use batch processing that had one primary problem which was high latency in data availability. This made a huge problem for time sensitive operations like fraud detection, healthcare monitoring or predictive maintenance, where real-time responsiveness was increasingly demanded.

While in the current era, data engineering has significantly evolved due to the rise of big data and cloud computing technologies as far as the mid 2000s. Modern systems can now handle structured, semi-structured and unstructured data at the large scale. Apache Hadoop framework is introduced that allows for the distributed processing of large data sets across clusters of computers using simple programming models. According to Azeroual & Fabre (2021) Hadoop characteristic includes can handle both structured and unstructured data, whether with a scheme or without a scheme, and handle large amounts of data in the range of terabyte to petabyte. In addition, Apache Kafka is a great tool for distributed data streaming. Kafka ingests streaming data from upstream sources and stores it durably in a distributed cluster, serving as a fault-tolerant buffer for downstream processors like Flink or Spark (Wu et al., 2020). These tools that provide automated data pipelines and streaming real-time data processing are essential for applications like machine learning and business intelligence.

Future developments in data engineering are fully expected to focus on AI-driven, automated, and platform-centric architectures. AI applications in data engineering have transformed traditional approaches through machine learning algorithms that automate pattern recognition and anomaly detection (Kovvuri, 2024). In addition, Natural Language Processing or rather NLP has integrated with the data management which enables automated metadata generation and semantic matching of disparate data sources. This will play a huge role in automating data pipeline design, monitoring and optimization in the organization. These AI-driven approaches will help with the improvement especially in data matching accuracy compared to traditional rule-based systems.

IV. Technologies Used & case examples

The selection of technology is one of the key factors that support data engineering lifecycle. It includes data ingestion, storage, processing, and serving (Fundamentals of Data Engineering, 2022). There are slight differences between traditional and modern technologies that have been utilized. Traditional data engineering typically relies on on-premises infrastructure, batch processing, and a rigid ETL pipeline. The data is transformed before it is loaded into the database. Meanwhile, modern data engineering is defined from cloud, separation of storage and compute, real-time streaming, and ELT paradigm. The data is loaded in some cheap data lakehouse before being transformed.

1. Storage and Warehousing

Traditional data engineering using centralized systems for reporting and analytics. As we know, a centralized database is a database system where all relevant data, such as

transaction and communication data, is stored in a single location to reduce redundancy and simplify management (Taylor & Francis, n.d.). Examples such as Teradata, Oracle Exadata, IBM Netezza. In addition, traditional methods used Relational Databases such as Microsoft SQL Server, Oracle DB, and MySQL as operational stores and smaller-scale data marts. Different from Modern data engineering, it utilizes the function of a decentralized database where it manages SQL engines using Cloud Data Warehouse. It is highly scalable where compute and storage scale independently. Snowflake and Google BigQuery are some of the examples. Also, the storage is not limited as Azure Data Lake Storage takes place for structured and unstructured data.

2. Processing Data

Early era of traditional data engineering, the Hadoop Ecosystem being the main wave of “big data”. It is a powerful open-source framework built to process and store massive datasets by distributing them across clusters of affordable and commodity hardware. Including 4 core modules which are HDFS(Hadoop Distributed File System) for storage system, MapReduce for processing, and YARN (Yet Another Resource Negotiator) for source management. As time moves forward, the modern method as mentioned before, the store and compute is separate, means the in-memory processing is significantly faster than Hadoop’s MapReduce. The utilization of Apache Spark enhances the uploads of data into memory at super high speeds (Staff, 2026). It gives a huge impact in data processing because faster performance allows for finishing the task such as querying and algorithmic functions.

3. Data Integration

Extract, Transform, Load (ETL) used in older or traditional methods. IBM DataStage and Informatica PowerCenter are examples of tools to perform the ETL. However, compared to modern data engineering, Extract, Load, Transform (ELT), ETL is slower because it includes a preliminary transformation step before loading data into the target. Thus, it becomes difficult to scale and slows performance as data size grows (Bartley & Bartley, 2025). ELT however is the opposite of ETL. It sends the raw data directly to the warehouse of the system. There, the transformation happens making the delivery of data ingestion faster.

V. Advantages and Disadvantages of Data Engineering

Data engineering offers foundational advantages for organizations transitioning from traditional RDBMS toward AI-driven analytical operations. The integration of these advanced technologies will give more impact to the discovery of hidden patterns, while also enhancing decision-making accuracy. However, despite these benefits, significant architectural and operational drawbacks remain unresolved, presenting critical topics for future research.

A. Advantages

1. Better Decision Making

Using data engineering enables better decision making. Being able to organize and process a large amount of data, organizations are able to generate a meaningful insight operational decision. High quality data means better financial performance and cost reduction, enhanced operational efficiency and better compliance.

2. Real-Time Data Processing

With the use of streaming technologies like Apache Kafka, data can be processed as it is generated, allowing organizations to respond quickly to events. This is particularly important in applications such as financial transactions, online services, and monitoring systems.

B. Disadvantages

1. High Cost

High cost usually associated with infrastructure, tools, and maintenance. Implementing data engineering solutions often requires significant investment in hardware, cloud services, and specialized software.

2. Complexity

Data engineering systems are also complex to design, implement and also integrate. For example, building efficient data pipelines involves integrating multiple components, which increases the risk of errors and system failures if not managed properly.

VI. Challenges and Future Trends

While modern tools solve old problems, they introduce new complexities. There might be a crisis of data modeling and technical debt. As organizations move faster with ELT, many skip the disciplined data modeling phase. A 2026 State of Data Engineering Survey found that nearly 90% of practitioners report pain points in data modeling, with "pressure to move fast" and "lack of clear ownership" being the primary culprits. This leads to "data swamps" where data is available but its meaning is undocumented or inconsistent (Reis, 2026). In addition, while managing distributed architecture across multi-cloud environments, there are chances that the pipelines will produce incorrect data even though the run processing is successful. This is called "silent failures" that will create massive demand for Data Observability (Fedynyshyn, 2025).

To hope for the best in the future, the field is shifting from "building pipelines" to "managing autonomous data ecosystems. The evolution from simple AI assistant to Agentic AI is what that has been looking for. These are autonomous systems that don't just suggest code but can detect a schema change in a source system, adapt the downstream pipeline, and alert stakeholders without human interaction. Not to mention, large enterprises are moving away from centralized "data kingdoms" toward a Data Mesh. In this decentralized model, specific business domains such as Marketing or Financing, own their data where known as "product",

while a central data engineering team provides the self-service infrastructure and universal governance standards (Fedynyshyn, 2025).

VII. Recommendations

Some changes or improvements need to be made to enhance the quality of data engineering itself. Due to the lack of structured modeling, this might lead to “data swamps”. Faster ingestion of ELT most likely leads to this. So, tools like dbt or Dataform can be used. As we know it can enforce version-controlled SQL modeling immediately after loading raw data. Moreover, the high latency of traditional method processing is no longer acceptable for modern methods. Using Kappa Architecture model can solve this problem. It allows users to utilize real-time data and historical data through a single processing logic. The impact can reduce the complexity of managing two separate systems.

VIII. Conclusion

In conclusion, data engineering has changed a lot from traditional database systems and batch processing to modern cloud-based and real-time technologies. Tools like Hadoop, Spark, and Kafka help organizations handle large amounts of data more efficiently and make faster decisions. However, there are still challenges such as high cost, system complexity, and data quality issues. The shift to faster methods like ELT can also cause problems like poor data organization if not managed properly. In the future, data engineering is expected to become more automated with the help of AI and new approaches like Data Mesh. Overall, data engineering is very important for helping organizations make better decisions, but it must be managed carefully to avoid problems.

IX. References

1. Azeroual, O., & Fabre, R. (2021). Processing big data with apache hadoop in the current challenging era of COVID-19. *Big Data and Cognitive Computing*, 5(1), 12.
<https://www.mdpi.com/2504-2289/5/1/12>
2. Bartley, K., & Bartley, K. (2025, May 28). *What is the Difference Between ETL and ELT?* Rivery. <https://rivery.io/blog/etl-vs-elt/>
3. Fedynyshyn, R. (2025, September 17). *7 data engineering trends for 2026 and beyond*. Software Development Company - N-iX. <https://www.n-ix.com/data-engineering-trends>
4. Hegde E. (2024). Evolution of Data Engineering in Modern Software Development, *Journal of Sustainable Solutions*, 1(4), 116-130. DOI:
<https://doi.org/10.36676/j.sust.sol.v1.i4.43>
5. Kovvuri, V. K. R. (2024). The role of AI in data engineering and integration in cloud computing. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, 10(6), 616-623.
https://www.academia.edu/download/123478403/IJSRCSEIT_Published_Artical.pdf
6. Reis, J. (2026, February 1). *2026 State of Data Engineering Survey*. Practical Data Community. https://joereis.github.io/practical_data_data_eng_survey/
7. Reis, J., & Housley, M. (2022). *Fundamentals of data engineering*. " O'Reilly Media, Inc.". <https://freecomputerbooks.com/books/Fundamentals-of-Data-Engineering.pdf>
8. Rucco, C., Longo, A., & Saad, M. (2025). Efficient data ingestion in cloud-based architecture: A data engineering design pattern proposal. *arXiv*.
<https://doi.org/10.48550/arXiv.2503.16079>
9. Staff, C. (2026, February 11). *What is Apache Spark? how it works and why it's used*. Coursera. <https://www.coursera.org/articles/apache-spark>

10. Taylor & Francis. (n.d.). *Centralized database - Knowledge and References* | Taylor & Francis.

https://taylorandfrancis.com/knowledge/Engineering_and_technology/Computer_science/Centralized_database/

11. Wu, H., Shang, Z., & Wolter, K. (2020, June). Learning to reliably deliver streaming data with apache kafka. In 2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN) (pp. 564-571). IEEE.

https://www.researchgate.net/profile/Han-Wu-28/publication/343353088_Learning_to_Reliably_Deliver_Streaming_Data_with_Apache_Kafka/links/5f4fa8cf299bf13a3197af51/Learning-to-Reliably-Deliver-Streaming-Data-with-Apache-Kafka.pdf